

SPinSynth-T Architecture Description

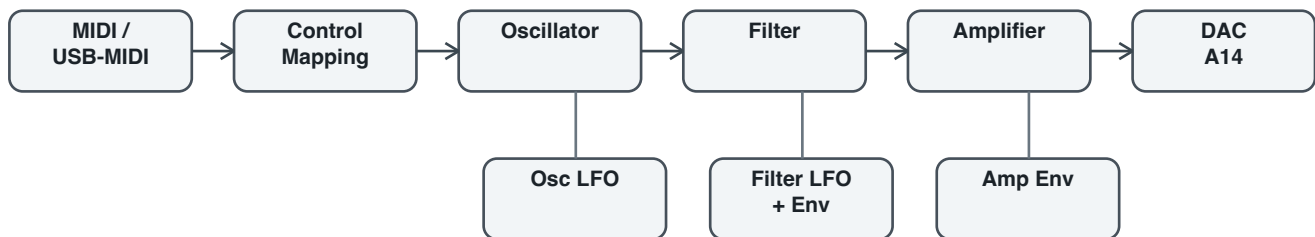
Initial architecture note for the Teensy 3.1/3.2 monophonic virtual analog synthesizer

Purpose

SPinSynth-T is a monophonic virtual analog synthesizer developed by Ricardo Peculis for Teensy 3.1/3.2. Unlike a PJRC Audio Library patch, the instrument implements its own oscillator, filter, amplifier, envelope generators, low-frequency oscillators, fixed-point helpers, double-buffered audio flow, and MIDI control mapping.

This document describes the current cleaned architecture and provides a starting point for future refactoring into an SPinSynthT voice class suitable for HMI control and later polyphony.

High-Level Signal Flow



- MIDI and USB-MIDI callbacks translate note, pitch bend, and control-change messages into synth parameters.
- The oscillator renders an audio buffer, the filter processes that buffer, and the amplifier applies the amp envelope and volume.
- A timer-driven DAC callback outputs one sample at a time from the amplifier buffer while the control/update timer prepares the next buffers.

Core Modules

Module	Role	Important Features
SPinSynth-T.ino	Hardware shell and control coordinator.	MIDI callbacks, CC mapping, timers, DAC output, buffer phase control.
Oscillator	Primary sound generator.	Sine, saw, square/pulse, XTREME; PolyBLEP saw edges; pulse from two corrected saws; LFO, pitch bend, tuning, portamento.
Filter	Virtual analog low-pass filter stage.	Cutoff/resonance smoothing, LFO cutoff modulation, envelope cutoff modulation, fixed-point buffer processing.
Amplifier	Amplitude stage after the filter.	Amp envelope, velocity scaling, master volume, fixed-point buffer processing.
EnvelopeGenerator	ADSR envelope engine.	Exponential attack/decay/release constants, gate/trigger model, multi-note on-count support, LED gate feedback.
LowFrequencyOscillator	Modulation oscillator.	Sine, saw, inverted saw, square/pulse; pulse width support; used by oscillator and filter.
SynthUtilities	Shared utility layer.	FixedPoint helpers, KeyEvent state, DoubleBuffer audio buffering.

Audio Timing And Buffers

- The active audio rate is 44.1 kHz, with a buffer size of 128 samples.
- The DAC timer calls `outputBufferedDAC()` at audio rate and writes to the Teensy DAC output pin A14.
- Three `DoubleBuffer` instances carry oscillator, filter, and amplifier data.
- The update phase alternates between generating oscillator samples and processing filter/amplifier buffers.
- `DoubleBuffer::read()` now returns the current sample before advancing, preserving sample zero after each swap.

Oscillator Architecture

The oscillator is the main identity of `SPinSynth-T`. It combines wavetable lookup, MIDI pitch tracking, pitch bend, master tuning, portamento, oscillator LFO, and multiple waveform modes.

- SAW uses `getSawSample()`, which applies PolyBLEP correction around the discontinuity.
- `SQUARE_PULSE` is generated by subtracting two PolyBLEP-corrected saws; the second phase controls pulse width.
- XTREME mixes main saw, detuned saw, sub saw, and pulse components, each controlled by MIDI CC amount/factor parameters.
- `getNaiveSample()` and `updateNaiveBuffer()` are intentionally retained as historical comparison paths without PolyBLEP.

Filter, Envelope, And Amplifier

- The filter is a four-stage virtual analog style low-pass with resonance feedback and stability limits on cutoff/resonance.
- Filter cutoff can be controlled directly, modulated by the modulation wheel, modulated by an LFO, and shaped by a filter envelope.
- The envelope generator uses exponential coefficient calculations inspired by analog-style ADSR behavior.
- The amplifier uses its own envelope and combines envelope level, velocity, and master volume in the final buffer stage.

MIDI And Control Model

The current sketch maps Arturia KeyLab-style MIDI controls directly to synth parameters. This is effective for the original MIDI-controlled instrument, but future HMI work should place a parameter layer between input devices and the synth engine.

- Note on/off drives oscillator key-event tracking and both filter/amplifier envelopes.
- Pitch bend is applied in the oscillator frequency path.
- Mod wheel can be assigned to filter cutoff, filter resonance, filter LFO amount, or oscillator LFO amount.
- Control changes set oscillator waveform, LFO parameters, filter ADSR, amplifier ADSR, portamento, tuning, and XTREME component amounts.

Current Architectural Strengths

- The synth engine is handmade and understandable: oscillator, filter, amplifier, envelopes, and LFOs are all project-owned code.
- The oscillator has a distinctive identity, especially through XTREME and the PolyBLEP pulse-from-saw approach.
- The monophonic key-event model already resembles the logic needed for a future voice allocator.
- The recent cleanup has reduced debug debris, removed unused alternate APIs, and clarified the current signal path.

Known Limitations And Cleanup Targets

- Global instances in the sketch should eventually become members of an SPinSynthT class.
- The sketch still mixes hardware, MIDI mapping, parameter interpretation, and audio scheduling responsibilities.
- Some headers still contain historical fixed-point/table-BLEP comments that should be reviewed carefully, not mass-removed.
- Some methods accept DoubleBuffer by value; changing this previously affected behavior, so that area should remain conservative.
- The current output is Teensy 3.x DAC-oriented; Teensy 4.x and PJRC Audio integration will need a different output strategy.

Proposed Next Architecture: SPinSynthT Class

The next major refactor should introduce a single class representing one complete synth voice. For the monophonic HMI project, one instance is enough. For a future polyphonic instrument, the same class can become the per-voice unit managed by a higher-level allocator.

- `begin(sampleRate, bufferSize)`
- `noteOn(pitch, velocity)`
- `noteOff(pitch, velocity)`
- `pitchBend(bend)`
- `controlChange(number, value)` or `setParameter(id, value)`
- `updateControl()`
- `render(outputBuffer)`

Recommended Future Split

- SPinSynthT owns Oscillator, Filter, Amplifier, envelopes, LFOs, and per-voice state.
- The sketch or application shell owns MIDI, HMI scanning, LCD updates, presets, timers, and audio output.
- A Parameter Manager translates MIDI, encoders, presets, and future automation into the same parameter model.
- A future VoiceAllocator can own several SPinSynthT instances for polyphony.

Future PJRC Audio Library Integration

The current best hybrid direction is to preserve SPinSynth-T as the source of oscillator/filter/envelope character, while using PJRC Audio Library where it provides leverage: effects, audio routing, audio output, and possibly MIDI/USB helpers.

- Short term: keep the engine independent while improving module boundaries.
- Medium term: compare SPinSynth-T oscillator output against PJRC bandlimited oscillators using ears and oscilloscope.

- Long term: consider wrapping the mixed SPinSynth-T output as a custom PJRC AudioStream object.

Prepared as an initial architecture note during the SPinSynth-T cleanup collaboration. This is intentionally a living document and should evolve as the SPinSynthT class emerges.